

텍스트 데이터 해독의 마스터키, 정규 표현식

파이썬 re 모듈과 NLTK를 활용한 자연어 전처리 파이프라인 구축

```
$%#@k!a^b*c&1(0)d<e>f|g~h/i\j{k}l_m+n-o=p  
$%#@k!a^b*c&1(0)d<abc|~h/i\j{k}l_m+n-o=p  
$%#@k!a^b*c&1(0)d<e>f|g~h/i\j{k}l_m+n-o=p
```


정규 표현식(Regular Expression)의 본질

도입 배경

- 자연어 처리(NLP) 전처리 과정에서 **노이즈가 섞인 텍스트 정제**는 필수적 과제입니다.
- 파이썬의 **re 모듈**은 복잡한 텍스트에서 **특정 규칙을 빠르게 찾아내는 강력한 무기**입니다.

핵심 원리

- 텍스트를 단순한 '**단어**'가 아닌 '**패턴**'으로 인식하는 **논리적 매칭 언어**입니다.
- 미리 컴파일된 **패턴(re.compile)**을 통해 대용량 데이터에서도 **압도적인 검색 속도**를 제공합니다.



암호 해독가의 터미널: 시각적 가이드

효율적인 패턴 인식 학습을 위해 본 강의에서는 일관된 색상 코드를 엄격히 적용합니다.



정규식 패턴 (탐색할 규칙)

[Pattern] `ab*c`



대상 텍스트 (검색 대상 원본)

[Text] `abbbc`



매칭 결과 (규칙에 의해 선택된 부분)

[Match] `abbbc`

기호 정의: . (점)

줄바꿈 문자(\n)를 제외한 한 개의 임의의 문자를 나타냅니다.



```
r = re.compile('a.c')
```

1	kkk	→	매치 실패
2	abc	→	[Match] abc
3	akc	→	[Match] akc
4	a5c	→	[Match] a5c

기호 정의: ? (물음표)

앞의 문자가 존재할 수도 있고, 존재하지 않을 수도 있습니다. (0개 또는 1개)

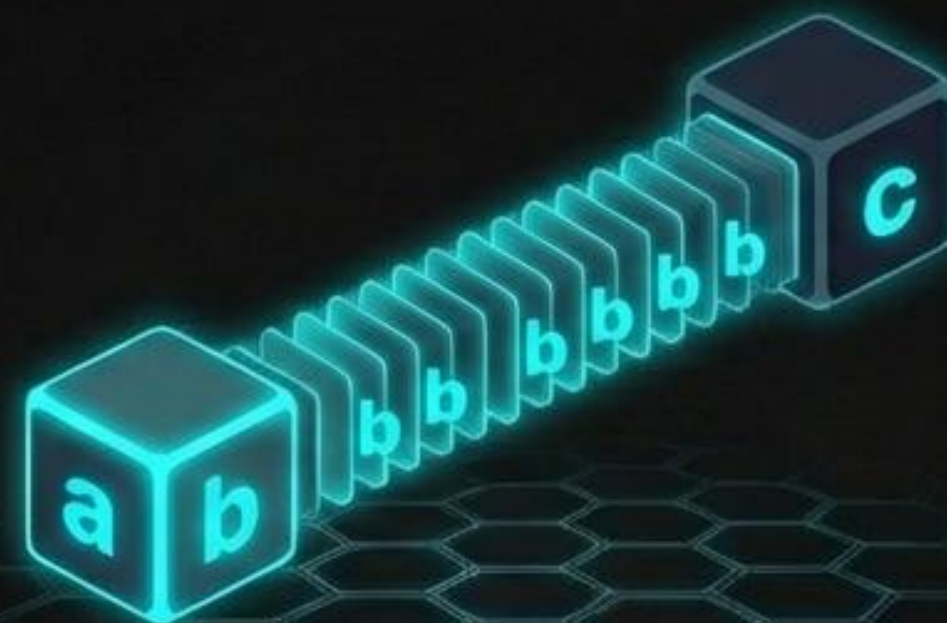


```
r = re.compile('ab?c')
```

1	abbc	→	매치 실패 (b가 2개)
2	abbc	→	매치 실패 (b가 2개)
3	abc	→	[Match] abc
4	ac	→	[Match] ac

기호 정의: * (별표)

앞의 문자가 존재하지 않을 수도 있고,
무한개로 존재할 수도 있습니다. (0개 이상)

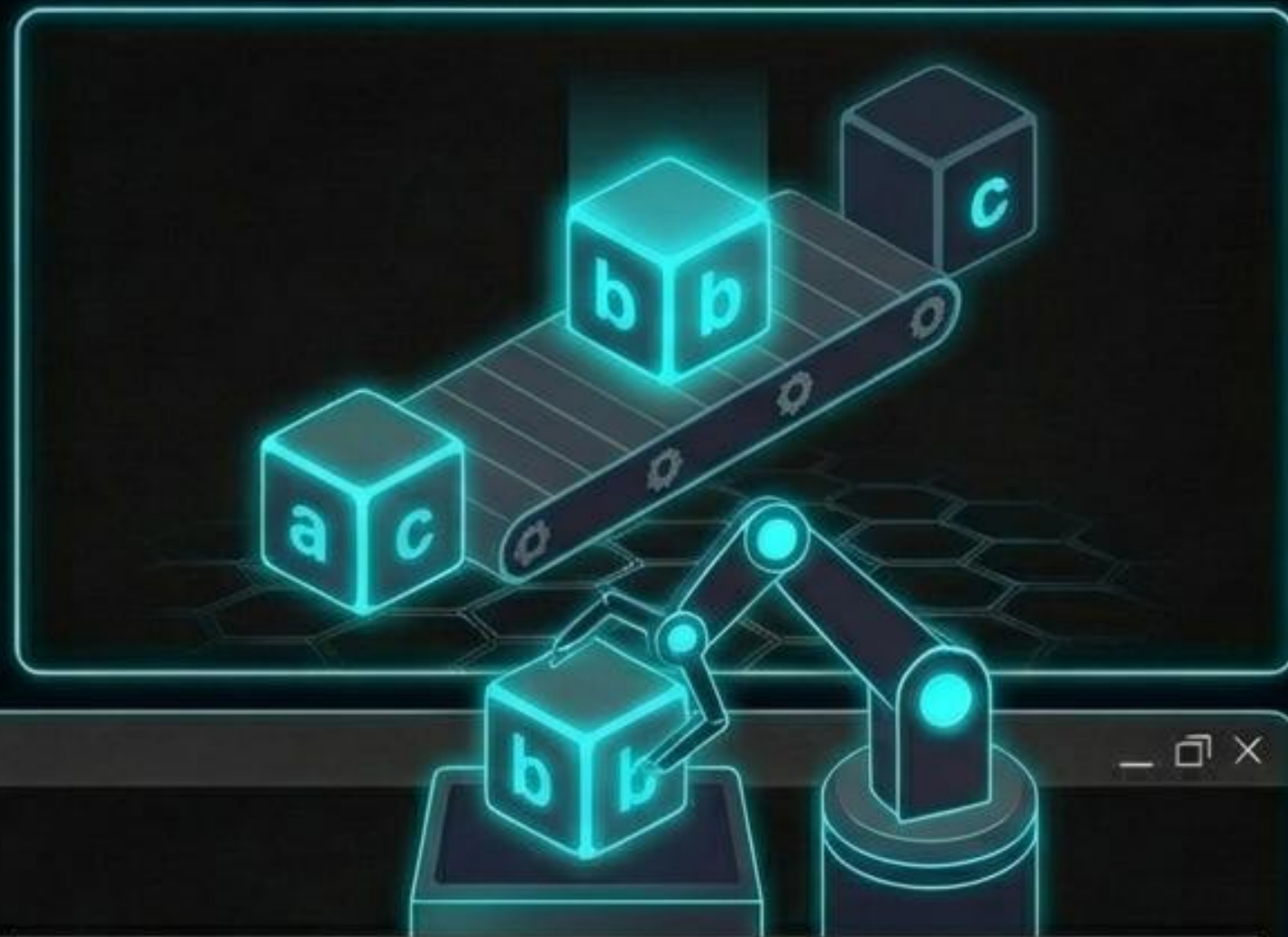


```
r = re.compile('ab*c')
```

1	a	→	매치 실패 (c가 없음)
2	ac	→	[Match] ac (b=0)
3	abc	→	[Match] abc (b=1)
4	abbbbc	→	[Match] abbbbc (b=4)

기호 정의: + (더하기)

앞의 문자가 최소 한 개 이상
존재해야 합니다. (1개 이상)



```
r = re.compile('ab+c')
```

1	ac	→	최소 1개 필수
2	ac	→	매치 실패 (b가 0개이므로 *와 달리 실패)
3	abc	→	[Match] abc (b=1)
4	abbbbc	→	[Match] abbbbc (b=4)

수량자(Quantifiers) 완전 정복 매트릭스

대상 텍스트	[Pattern] ab?c	[Pattern] ab*c	[Pattern] ab+c
ac (b=0)	[Match] ac	[Match] ac	실패 ✕
abc (b=1)	[Match] abc	[Match] abc	[Match] abc
abbc (b=2)	실패 ✕	[Match] abbc	[Match] abbc

기호 정의: ^ (캐럿)

뒤의 문자열로 문자열이 '시작'됨을 지정합니다. (문자열의 입구를 고정)



```
r = re.compile('^ab')
```

1	bbc	→	매치 실패
2	zab	→	매치 실패 (ab가 있지만 시작이 아님)
3	abz	→	[Match] abz

기호 정의: {숫자} (중괄호 단일)

해당 문자를 정확히 숫자만큼
반복한 것만 매치합니다.

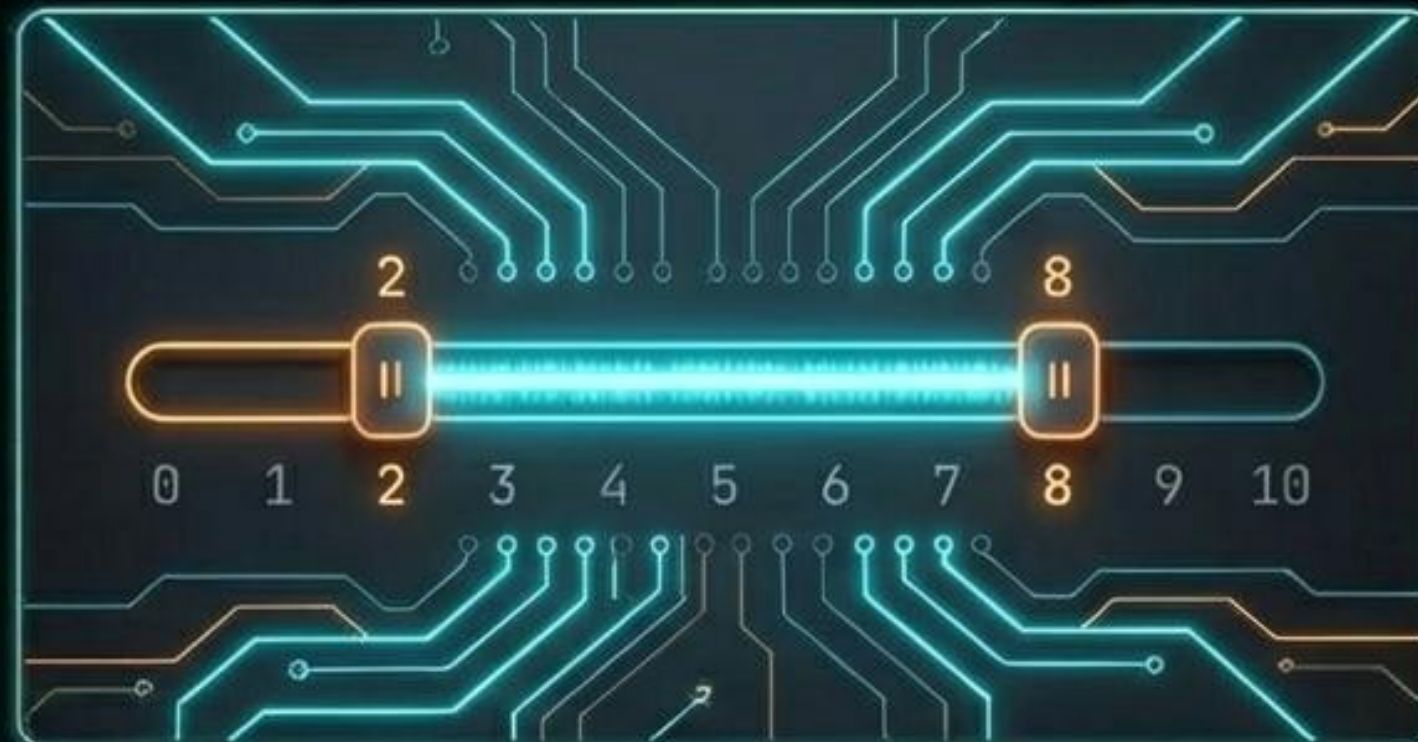


```
r = re.compile('ab{2}c')
```

1	ac	→	매치 실패 (b=0)
2	abc	→	매치 실패 (b=1)
3	abbbbbc	→	매치 실패 (b=5)
4	abbc	→	[Match] abbc

기호 정의: {min, max} (중괄호 범위)

- {숫자1, 숫자2}: 최소 숫자1 이상, 최대 숫자2 이하 반복.
- {숫자,}: 최소 숫자 이상 무한 반복.



CODE EXECUTION

[Pattern] ab{2,8}c

abc	→	매치 실패
abbbbbbbbbc	→	매치 실패 (b=9개)
abbbbbbbbbc	→	[Match] abbbbbbbbc
	→	[Match] abbbbbbbbc

[Pattern] a{2,}bc

aabc	→	[Match] aabc
aaaaaaaaabc	→	[Match] aaaaaaaaaabc

기호 정의: [] (대괄호)

- 대괄호 안의 문자들 중 '한 개의 문자'와 매치합니다.
- 하이픈(-)으로 범위를 지정하여 알파벳이나 숫자를 쉽게 표현할 수 있습니다.



CODE EXECUTION

[Pattern] [abc]

zzz → 매치 실패

baac → 매치 결과: [b]aac

[Pattern] [a-z]

AAA → 매치 실패

[a]BC → 매치 결과: [a]BC

기호 정의: [^문자] (대괄호 안의 캐럿)

- 해당 문자를 '제외한' 모든 문자를 매치합니다.
(선택이 배제로 반전됨)
- 주의: 대괄호 밖의 ^ (시작점)와 역할이
완전히 다릅니다.



CODE EXECUTION

```
r = re.compile('[^abc]')
```

a	→	매치 실패 (배제 대상)
ab	→	매치 실패 (배제 대상)
d	→	[Match] d
1	→	[Match] 1

알파벳의 결합: 자주 쓰이는 축약 표현 (Macros)

복잡한 대괄호 구문이 실무에서 가장 많이 쓰이는 역슬래시 단축어로 압축됩니다.

[0-9]	→	\d	모든 숫자 매치
[^0-9]	→	\D	숫자 제외 매치
[\t\n\r\f\v]	→	\s	모든 공백 매치
[a-zA-Z0-9]	→	\w	문자 및 숫자 매치

⚠ 규칙: 대문자(예: \D, \S, \W)는 항상 소문자의 반대(Negation) 역할을 수행합니다.

함수 활용: `re.match()` vs `re.search()`

문자열을 탐색하는 두 가지 동사의 결정적 차이

`re.match()` (문지기 - Gatekeeper)

문자열의 '시작 부분'부터 패턴을 검사.
입구에서 틀리면 즉시 탐색 포기.



kkkabc
ab.

`r.match('kkkabc')`
실패 (시작이 k이므로 포기)

`re.search()` (레이더 - Radar)

문자열 '전체'를 훑어서 패턴이
중간에 있더라도 끝까지 찾아냄.



kkkabc
ab.

`r.search('kkkabc')`
"kkk" + [abc]

함수 활용: `re.split()` - 패턴을 도끼로 사용하기

복잡한 정규 표현식을 기준으로 문자열을 분할하여 리스트 블록으로 쪼갭니다.

사과+딸기+수박+메론+바나나

['사과', '딸기', '수박', '메론', '바나나']

텍스트: 사과+딸기+수박+메론+바나나

패턴: `\+`

명령어: `re.split('\+', text)`

결과: `['사과', '딸기', '수박', '메론', '바나나']`

함수 활용: `re.findall()` - 원하는 데이터만 쓱쓱 뽑아내기

정규 표현식과 매치되는 모든 경우를 찾아 리스트 바구니에 담습니다.



패턴: `\d+` 숫자 1개 이상이 연속된 덩어리

명령어: `re.findall('\d+', text)`

결과: `['010', '1234', '1234', '30']` 문자를 완벽히 배제하고 숫자만 추출

함수 활용: `re.sub()` - 노이즈 제거와 대체

정규식 패턴과 일치하는 부분을 찾아 다른 문자열(예: 공백)로 치환(Substitute)합니다.

regex or regexp[1]
(sometimes called...)[2][3]

[^a-zA-Z]

regex or regexp
sometimes called

패턴: `[^a-zA-Z]` { 알파벳 대소문자를 제외한 모든 문자 = 노이즈
명령어: `re.sub('[^a-zA-Z]', ' ', text)`
결과: 알파벳 외의 괄호와 숫자가 모두 공백으로 대체됨

실전 응용: 텍스트 테이블 정제 파이프라인

엉망인 텍스트 데이터에서 공백 분리부터 고유명사 추출까지

```
원본: 100 John PROF \  
      101 James STUD \  
      102 Mac STUD
```

Step 1: 분할 (Split)

```
re.split('\s+', text)
```

← \s+ (1개 이상의 공백)을
기준으로 단어 단위 쪼개기

```
['100', 'John', 'PROF', '101', 'James', 'STUD', '102', 'Mac', 'STUD']
```

Step 2: ID 추출

```
re.findall('\d+', text)
```

← 숫자만 추출

```
['100', '101', '102']
```

Step 3: 이름 추출

```
re.findall('[A-Z][a-z]+', text)
```

← 패턴 분석: [A-Z](첫글자 대문자)
+ [a-z]+(소문자 1개 이상 연속)

```
['John', 'James', 'Mac']
```


실전 응용: NLTK RegexpTokenizer 실습

자연어 처리 라이브러리를 활용한 정규표현식 기반 단어 토큰화(Tokenization)

대상 텍스트: Don't be fooled by the dark sounding name, Mr. Jone's...

접근 1: 문자/숫자만 추출

```
RegexTokenizer('[\w]+')
```



```
['Don', 't', 'be', ...  
'Mr', 'Jone', 's']
```

결과 분석: 아포스트로피(')와 온점(.)이 제거되고 완벽하게 단어만 분리됨

접근 2: 공백을 기준으로 쪼개기

```
RegexTokenizer('\s+', gaps=True)
```



```
["Don't", 'be', ...  
'name,', 'Mr.', "Jone's"]
```

결과 분석: 구두점이 단어에 붙어 그대로 보존된 채 토큰화됨

암호 해독가에서 NLP 엔지니어로

복잡한 기호의 나열은 이제 논리적인 조립식 블록이 되었습니다.

[기호] (Syntax)

문자의 종류와 수량, 위치를 정의하는 엄격한 규칙.

[re 모듈] (Engine)

탐색(search), 분할(split), 추출(findall), 치환(sub)을 수행하는 엔진.

[NLTK] (Application)

기계가 이해할 수 있는 토큰(Token)으로 자연어를 규격화하는 실전 무기.

○○○

— □ ×

PREPROCESSING COMPLETE _

ID	Name	Role	Statement
100	John	PROF	Profesor
101	James	STUD	Namring
102	Mac	STUD	Growing